

Quy tắc viết code – Coding Conventions

Tên App/Project

Quy ước: Sử dụng **PascalCase** (viết hoa chữ cái đầu của mỗi từ). Tên phải ngắn gọn, dễ hiểu và phản ánh đúng chức năng của dự án. Không sử dụng ký tự đặc biệt hoặc khoảng trắng.

Ví dụ: StudentManagement, LibraryManagement, PetCareSystem, FacebookMessenger

Tên Package/Thư mục chứa code

Quy ước: Sử dụng **camelCase** hoặc **lowercase**. Tên thư mục ngắn gọn, không dấu, không khoảng trắng và thể hiện đúng chức năng.

Ví dụ: controllers, models, routes, middleware, components, contexts, utils, hooks

Tên Class / Interface

Quy ước: Sử dụng **PascalCase**. Tên là danh từ, rõ nghĩa, không viết tắt.

Ví dụ: Student, Course, Faculty, User, AttendanceSession, AuthContext

Tên Biến / Thuộc tính / Dữ liệu

Quy ước biến thường: Sử dụng **camelCase**. Tên biến phải thể hiện rõ ý nghĩa và mục đích sử dụng.

Ví dụ: studentName, courseCode, isLoading, modalVisible, selectedFaculty

Quy ước hằng số (const, final): Viết hoa toàn bộ, các từ nối bằng dấu gạch dưới (_).

Ví dụ: API_URL, MAX_FILE_SIZE, DEFAULT_ROLE, JWT_SECRET

Tên Hàm/Phương Thức

Quy ước: Đặt theo cấu trúc **Verb + Noun** (Động từ + Danh từ). Sử dụng **camelCase**.

Ví dụ: getStudent(), createCourse(), updateProfile(), deleteFaculty(), fetchCourses(), resetCourseForm()

Khuyến khích: Tên hàm phản ánh đúng chức năng thực hiện.

Indentation / Thụt lề cho code

Quy ước: Mỗi cấp lệnh được thụt vào **4 khoảng trắng** (hoặc 1 Tab). Luôn sử dụng thống nhất trong toàn bộ dự án.

Ví dụ:

```
if (isLoading) {  
    return;  
}
```

Quy ước: Có khoảng trắng sau từ khóa và quanh toán tử.

Ví dụ: `const total = price + tax;`

Comment

Quy ước: Chỉ comment khi cần giải thích **mục đích** hoặc **lý do**. Không comment những gì code đã thể hiện rõ.

Quy ước Backend: Sử dụng **JSDoc** cho Function, Class hoặc Module công khai.

Ví dụ (JSDoc):

```
/**  
 * Lấy danh sách sinh viên theo khoa.  
 *  
 * @param {string} facultyId - Mã khoa.  
 * @returns {Promise<Array>} Danh sách sinh viên.  
 */  
async function getStudentsByFaculty(facultyId) {  
    ...  
}
```

Ví dụ (React Component dùng JSDoc):

```
/**  
 * Hiển thị danh sách khóa học của sinh viên.  
 *  
 * @returns {JSX.Element}  
 */  
export default function CourseList() {  
    ...  
}
```

Sử dụng **JSDoc** cho các **Function**, **API**, **Utility** và **Component**.

Chỉ dùng comment thường (`//`) để giải thích **Business Logic** hoặc đoạn xử lý phức tạp.

Không comment những câu lệnh đơn giản hoặc dễ hiểu.

Validation – Reuse – Interaction

Tự quy tắc hàm và tái sử dụng: Tách các đoạn xử lý dùng nhiều lần thành Function hoặc Utility để tránh lặp code.

Ví dụ:

validateInput()

formatDate()

calculateTotal()

Kiểm tra dữ liệu nhập: Luôn kiểm tra dữ liệu trước khi lưu hoặc gửi lên Server.

Ví dụ:

- Không để trống.
- Đúng định dạng Email.
- Độ dài Password.
- Kiểm tra giá trị Number.

Tránh Crash chương trình: Luôn xử lý Exception, kiểm tra null, lỗi API và lỗi mạng trước khi thao tác dữ liệu.

Không lặp mã nguồn (DRY - Don't Repeat Yourself)

Quy ước: Không sao chép hoặc viết lại cùng một đoạn xử lý ở nhiều nơi. Những chức năng được sử dụng nhiều lần phải được tách thành Function, Utility, Hook hoặc Service để tái sử dụng.

Ví dụ:

```
// utils/formatDate.js
export function formatDate(date) {
  ...
}

import { formatDate } from "../utils/formatDate";
```

```
const createdAt = formatDate(student.createdAt);
```

Khuyến khích:

- Tái sử dụng Utility, Hook và Service.
 - Không copy cùng một đoạn code ở nhiều Component hoặc Controller.
 - Nếu một đoạn xử lý xuất hiện từ 2 lần trở lên, nên cân nhắc tách thành hàm riêng
-

Frontend tổ chức theo Context + Component

Quy ước: Giao diện được chia thành các Component nhỏ, mỗi Component chỉ đảm nhiệm một chức năng. Trạng thái dùng chung của toàn ứng dụng được quản lý bằng Context.

Cấu trúc:

src/

```
|— app/  
|— components/  
|— contexts/  
|— hooks/  
|— services/  
|— styles/  
|— utils/  
└— config/
```

Nguyên tắc:

- Component chỉ xử lý giao diện (UI).
 - Context quản lý State dùng chung.
 - API gọi thông qua Service.
 - Hàm dùng chung đặt trong Utility hoặc Hook.
 - Không viết xử lý nghiệp vụ phức tạp trong Component.
-

Áp dụng mô hình MVC cho Backend

Quy ước: Backend được xây dựng theo mô hình MVC (Model – View – Controller) nhằm tách biệt dữ liệu, xử lý nghiệp vụ và định tuyến, giúp dễ bảo trì và mở rộng.

Cấu trúc tham khảo:

backend/

```
|— controllers/  
|— middleware/  
|— models/  
|— routes/  
|— config/  
└— server.js
```

Nguyên tắc:

- **Model:** Quản lý dữ liệu và làm việc với Database.
- **Controller:** Xử lý Business Logic và trả kết quả.
- **Route:** Chỉ định tuyến Request đến Controller.
- **Middleware:** Kiểm tra Token, Validation, Logging, phân quyền,...
- Không viết Business Logic trực tiếp trong Route.

Ví dụ:

```
// routes/studentRoutes.js
router.get("/", studentController.getStudents);
// controllers/studentController.js
exports.getStudents = async (req, res) => {
  ...
};
```

Tổng hợp

Áp dụng mô hình **MVC** cho Backend.

Frontend tổ chức theo **Context + Component**.

Business Logic đặt trong **Controller**.

Không viết Logic trực tiếp trong Route.

Một hàm chỉ nên thực hiện **một chức năng**.

Không lặp mã nguồn (**DRY - Don't Repeat Yourself**).

Tên hàm theo chuẩn **Verb + Noun**.

Tên biến, hàm sử dụng **camelCase**.

Tên Class sử dụng **PascalCase**.

Hằng số sử dụng **UPPER_SNAKE_CASE**.

Comment theo chuẩn **JSDoc** khi cần thiết.